🔬 Deep-Dive 면접 대응 매뉴얼

> 14개 심층 질문 + 꼬리 질문. 각 답변에 **실제 코드 파일 경로**와 **설계 근거**를 포함.

Q1. 입력 파이프라인을 처음부터 끝까지 설명해 주세요.

전체 흐름 (10단계)

```
사용자 입력 (텍스트/이미지)
  ↓
① 인증 확인 — auth.api.getSession()
  ↓ (실패 → 401)
② 빈 입력 검증 — input.trim() 체크
  ↓ (빈 → 에러 반환)
③ OOD 필터 — isFinancialInput() [느슨한 선차단]
  ↓ (명백한 비도메인만 실패 → 안내 문구 반환)
④ 은행 메시지 전처리 — isBankMessage() → preprocessBankMessage()
  ↓ (잔액/한도/할부 텍스트 노이즈 제거)
⑤ 모델 라우팅 — resolveTextProvider()
  ↓ (Fireworks 3회 → Kimi 전환)
⑥ 사용자 데이터 병렬 조회 — Promise.all([카테고리, 계정])
  ↓
⑦ LLM 호출 — parseUnifiedText() / parseUnifiedImage()
  ↓ (타임아웃 25~75초, 재시도 1회)
⑧ JSON 추출 — extractJSON() (마크다운 코드블록 처리)
  ↓
⑨ 응답 검증 — parseUnifiedResponse() → validateTransactions() / validateAccounts()
  ↓ (OOD 거부 → rejected:true → Error throw)
⑩ 결과 반환 → 프론트에서 확인 시트 표시 → 저장 시 createTransactions()
```

관련 파일:

- `src/server/services/parse-core.ts` — ①~⑥ 오케스트레이션
- `src/server/llm/ood-filter.ts` — ③ OOD 필터
- `src/server/llm/bank-message.ts` — ④ 은행 메시지 전처리
- `src/server/llm/index.ts` — ⑦~⑨ LLM 호출 + 검증
- `src/server/actions/transaction.ts` — ⑩ DB 저장

꼬리: 인증 실패, 빈 입력, OOD는 각각 어디서 막나?

방어 계층	위치	코드
인증 실패	`parse-core.ts` / `route.ts`	`auth.api.getSession()` → null이면 즉시 반환
빈 입력	`parse-core.ts:140`	`if (!input.trim())` → 에러 반환
OOD	`parse-core.ts:145`	`isFinancialInput(input)` → false면 에러
OOD (2차)	`llm/index.ts:106`	LLM이 `{rejected: true}` 반환 → Error throw

꼬리: 왜 OOD를 완전히 LLM에만 맡기지 않나?

비용/속도: "오늘 날씨 어때?"처럼 명백한 비도메인까지 매번 LLM에 보내면 응답 지연과 비용이 늘어난다. **사용자 경험 우선순위**: 하지만 false negative가 더 치명적이므로, 선필터는 **명백한 잡담/날씨/코딩 요청** 정도만 차단하고 애매한 입력은 LLM 2차 필터로 넘긴다. **이중 방어**: 선필터는 아주 느슨하게, 최종 거부는 LLM의 `rejected: true` 와 확인 시트 취소 UX가 보완한다.

Q2. 왜 거래와 자산/부채를 통합 파서로 처리했나?

답변

사용자 관점에서 "거래 입력"과 "자산 등록"의 **입력 인터페이스가 동일**하기 때문이다.

- "점심 9000" → 거래
- "국민은행 잔액 150만원" → 자산

사용자가 "이건 거래고 이건 자산이야"를 구분할 필요 없이, **LLM이 intent를 자동 판별**한다.

실제 코드 (`llm/index.ts:100-129`):

```
function parseUnifiedResponse(parsed) {
    const intent = obj.intent === "account" ? "account" : "transaction";
    const transactions = validateTransactions(obj.transactions);
    const accounts = validateAccounts(obj.accounts);
    return { intent, transactions, accounts };
}
```

꼬리: 혼합 입력 시 UI 순서는?

`intent` 필드로 **주된 의도**를 판단한다. `intent: "transaction"` 이면 거래 확인 시트를 먼저 보여주고, `intent: "account"` 면 자산/부채 확인 시트를 먼저 보여준다. 현재 한 번에 둘 다 들어오는 경우는 intent 기준으로 하나만 처리하고, 나머지는 무시되는 구조다.

꼬리: 왜 시트를 나눠서 처리하나?

UX 안전성: 거래(금액+날짜+카테고리)와 자산(잔액+유형)은 확인/수정할 필드가 완전히 다르다. 하나의 시트에 넣으면 복잡도가 폭발한다. **원자적 저장**: 거래와 자산은 서로 다른 테이블 (`transactions` vs `accounts`)에 저장되므로, 함께 저장하면 부분 실패 시 롤백이 복잡해진다.

Q3. 현재 모델 라우팅 정책을 설명해 주세요.

답변 (`parse-core.ts:14-57`)

```
세션별 Fireworks 사용 카운터 (인메모리 Map)
  ↓
사용 횟수 < 3 → Fireworks (빠름, 미국 서버)
사용 횟수 ≥ 3 → Kimi 직접 호출 (정확, 한국 근접 서버)
  ↓
Fireworks 키 없음 → Kimi만 사용
```

```
Kimi 키 없음 → Fireworks만 사용 (제한 없이)
둘 다 없음 → 에러 반환
```

왜 세션별 3회 제한?: 신규 사용자가 처음 써볼 때 빠른 응답(Fireworks ≈ ~~12초)을 경험하게 하고, 이후에는 정확도가 높은 Kimi(≈ 3~~5초)로 전환. Fireworks는 동일 모델이지만 미국 서버라 latency는 낮고 비용도 약간 낮음.

꼬리: 인메모리 카운터의 운영 한계?

- **서버 재시작 시 리셋**: Vercel Serverless는 cold start 시 Map이 초기화됨 → 사용자가 추가 무료 횟수를 얻음 (허용 가능한 수준)
- **24시간 TTL**: `pruneStaleEntries()` 가 24시간 초과 엔트리를 자동 정리 → 메모리 누수 방지
- `MAX_MAP_SIZE = 1000` : Map이 1000개 넘으면 강제 정리

꼬리: 멀티 인스턴스 환경이면?

Vercel Serverless는 각 인스턴스가 독립 메모리 → Map이 공유되지 않음. 해결:

1. **Redis**: 세션 카운터를 Redis에 저장 (가장 일반적)
2. **DB**: `session` 테이블에 `fireworksUsage` 컬럼 추가
3. **현실적 판단**: 개인 가계부 앱에서 사용자 수가 적으므로, 인메모리도 충분. 과잉 설계를 피했다.

Q4. OOD 필터와 은행 메시지 전처리를 둘 다 둔 이유

답변

다른 목적:

- **OOD 필터** (`ood-filter.ts`): "이 입력이 가계부와 관련 있는가?" → 무관한 입력 **차단** (LLM 호출 자체를 안 함)
- **은행 메시지 전처리** (`bank-message.ts`): "관련은 있지만, 잔액/한도/할부 같은 **노이즈를 제거**해서 LLM이 더 정확하게 파싱하도록" → LLM 호출 전 입력을 **정제**

```
OOD 필터: "오늘 날씨 어때?" → 차단 (LLM 호출 X)
은행 전처리: "[카카오뱅크] 출금 5,500원 스타벅스 잔액 150,000원" → "카카오뱅크 출금 5,500원 스타벅스" (잔액 제거)
```

꼬리: 정규식 vs LLM의 경계?

정규식으로 처리	LLM에 맡김
OOD 판별 (키워드+금액패턴)	카테고리 분류 ("스벅" → 카페)
은행 메시지 노이즈 제거	자연어 날짜 해석 ("어제" → 2025-03-08)
은행 메시지 감지	금액 변환 ("9천" → 9000)
	의도 분류 (거래 vs 자산)

원칙: 규칙이 명확하고 변하지 않는 것 → 정규식. 문맥 이해가 필요한 것 → LLM.

꼬리: false positive vs false negative?

false negative(진짜 거래를 OOD로 막음)을 더 경계한다. 사용자가 정당한 입력을 했는데 "가계부와 관련 없습니다"라고 거부당하면 극도로 짜증남. 반면 false positive(무관 입력이 통과)는 LLM이 2차로 `rejected: true` 를 반환하거나, 최악의 경우 사용자가 확인 시트에서 취소하면 됨. → OOD 필터는 **느슨하게** 설정하고, LLM의 2차 필터로 보완.

Q5. 이미지 입력 처리에서 클라이언트 압축

답변

> **현재 이 프로젝트에는 클라이언트 측 이미지 압축 로직이 구현되어 있지 않다.**

사용자가 업로드한 이미지는 `File` 객체에서 `arrayBuffer()` 로 읽어 base64로 변환한 뒤 그대로 LLM API에 전송한다 (`route.ts:34-35`).

만약 구현한다면?:

- **클라이언트 압축 이유**: base64 인코딩은 원본 대비 ~33% 크기 증가. 10MB 사진 → 13MB base64 → 서버 업로드 시간 증가 + LLM 토큰 비용 증가
- **Canvas API**로 리사이즈 + JPEG 품질 80%로 압축 → 1~2MB로 축소
- **서버가 아닌 클라이언트에서**: 네트워크 대역폭 절약이 핵심. 서버에서 압축하면 이미 큰 파일이 업로드된 후라 의미가 줄어든다.
- **추가 UX**: 파일 크기 제한(5MB), 업로드 프로그레스 바, 취소 버튼 등은 현재 미구현이지만 필요한 개선점.

Q6. 카테고리 추가 흐름이 UI에도 저장 단계에도 있는 이유

답변

UI 측: 사용자가 확인 시트에서 "새 카테고리 추가" 버튼을 눌러 직접 생성 가능. **저장 측** (`transaction.ts:145-176`): LLM이 반환한 카테고리가 DB에 없으면 **자동으로 생성** (insert + onConflictDoNothing).

```
// 누락 카테고리 자동 보정
const missingMap = new Map();
for (const item of normalizedItems) {
    const key = categoryKey(item.type, item.category);
    if (!categoryMap.has(key)) {
        missingMap.set(key, { name: item.category, type: item.type });
    }
}
if (missingMap.size > 0) {
    await db.insert(categories).values([...]).onConflictDoNothing();
}
```

꼬리: 이중 방어가 필요한 실제 상황?

1. **UI에서 카테고리를 추가했지만, 네트워크 지연으로 DB 반영 안 됨** → 저장 시점에 다시 확인
2. **LLM이 suggestedCategory로 새 이름을 제안** → 사용자가 수락하면 즉시 저장해야 하는데, UI 생성과 저장 사이에 타이밍 갭 존재

꼬리: 중복 카테고리 방어?

`onConflictDoNothing({ target: [categories.userId, categories.type, categories.name] })` — DB 레벨의 unique constraint로 동일 사용자의 같은 type+name 카테고리는 절대 중복 생성되지 않음. 동시성 문제도 DB가 해결.

Q7. 고정 거래 중복 방지 로직

답변 (`transaction.ts:96-115, 209-245`)

```
function isRecurringDuplicate(candidate, existingList) {
    return existingList.some((row) => {
        if (row.type !== candidate.type) return false;
        if (row.amount !== candidate.amount) return false;
        if (normalizeDescription(row.description) !== normalizeDescription(candidate.description)) return false;
        if ((row.categoryId ?? null) !== (candidate.categoryId ?? null)) return false;
        return Math.abs(row.dayOfMonth - candidate.dayOfMonth) <= 1; // 날짜 ±1일 허용
    });
}
```

시그니처: `type + amount + description(정규화) + categoryId + dayOfMonth(±1일)`

꼬리: 왜 완전 일치만 보지 않았나?

`dayOfMonth` ±1일 허용 이유: "매달 15일 넷플릭스"를 입력했는데, 실제 결제일이 14일이거나 16일일 수 있음. 완전 일치만 보면 같은 구독을 중복 등록하게 됨. `normalizeDescription()` 도 공백/대소문자를 정규화해서 "넷플릭스"와 "넷플릭스 " 같은 미세 차이를 무시.

꼬리: TOCTOU란?

Time-Of-Check to Time-Of-Use: "확인 시점"과 "사용 시점" 사이에 데이터가 바뀌는 레이스 컨디션.

이 코드에서의 방어:

```
// db.transaction 안에서 조회+삽입을 원자적으로 수행
const savedCount = await db.transaction(async (tx) => {
    const existingRecurring = await tx.select(...); // 조회
    // ... 중복 판정 ...
    await tx.insert(recurringTransactions).values(...); // 삽입
});
```

`db.transaction()` 내부에서 조회와 삽입을 함께 하므로, 두 사용자가 동시에 같은 고정거래를 등록해도 트랜잭션 격리 수준에 의해 하나만 성공한다.

Q8. 거래와 계좌 잔액의 정합성

답변 (`transaction.ts:19-64`)

핵심 함수 2개:

- `adjustAccountBalance(tx, accountId, type, amount)` — 거래 생성 시 잔액 반영
- `reverseAccountBalance(tx, accountId, type, amount)` — 거래 삭제/수정 시 이전 값 역산

```
생성: income 5000 → 잔액 +5000
삭제: income 5000을 삭제 → 잔액 -5000 (역산)
수정: 기존 income 5000 → expense 3000으로 변경
  → 먼저 이전 값 역산 (잔액 -5000)
  → 그 다음 새 값 반영 (잔액 -3000)
```

꼬리: "이전 값 역산 후 새 값 반영" 구조가 왜 필요?

단순히 `newAmount - oldAmount` 차이만 반영하면 **type 변경(income→expense)**을 처리 못 한다. 역산+재반영이면 type이 바뀌어도, 금액이 바뀌어도, 계좌가 바뀌어도 모두 정확하다.

꼬리: 계좌를 바꾸는 수정에서의 버그 가능성?

```
// 이전 계좌 역산
await reverseAccountBalance(tx, existing.accountId, existing.type, existing.amount);
// 새 계좌 반영
await adjustAccountBalance(tx, newAccountId, newType, newAmount);
```

이전 계좌와 새 계좌가 다를 수 있다. 코드는 이를 정확히 처리: 이전 계좌에서 역산하고, 새 계좌에서 반영한다. 만약 하나만 했으면 한쪽 계좌의 잔액이 맞지 않게 된다.

꼬리: 트랜잭션 밖이면?

조회(`SELECT balance FOR UPDATE`) → 계산 → 저장(`UPDATE`) 사이에 다른 요청이 잔액을 변경하면 **lost update** 발생. `db.transaction()` + `FOR UPDATE` 잠금으로 원자성 보장. 이것이 없으면 동시 거래 입력 시 잔액이 어긋난다.

Q9. 현재 암호화 구현 수준 평가

답변 (`src/server/lib/crypto.ts`)

알고리즘: AES-256-GCM (인증 + 암호화 동시) **구조**: `v1:` prefix + base64(IV 12byte + AuthTag 16byte + ciphertext)

필드	암호화 여부	이유
`accounts.name` (계좌명)	✅ 암호화	금융 기관명 = 민감 정보
`accounts.balance` (잔액)	✅ 암호화	핵심 금융 데이터
`transactions.originalInput`	✅ 암호화	원문에 카드번호 등 포함 가능
`transactions.memo`	✅ 암호화	자유 텍스트 = 민감 가능성
`transactions.amount` (거래금액)	❌ 평문	SQL SUM/GROUP BY 필요
`transactions.description`	❌ 평문	검색(ILIKE)에 필요
`transactions.date`	❌ 평문	범위 쿼리(gte/lt)에 필요

꼬리: 남는 보안 리스크?

1. `amount` 가 평문 → DB 유출 시 "누가 얼마를 썼는지" 노출. 하지만 암호화하면 `SUM(amount)` 같은 집계가 불가능 → **동형 암호화** 또는 **애플리케이션 레벨 집계**가 필요 (비용/복잡도 대비 현실적이지 않음)
2. `ENCRYPTION_KEY` 가 환경변수 하나 → 키 로테이션 시 기존 데이터 복호화 불가. `KEY_VERSION = "v1"` prefix로 향후 키 버전 관리는 가능하도록 설계됨

꼬리: 다음으로 암호화할 필드?

`description` — "스타벅스 강남점" 같은 상호명은 개인 행동 패턴을 드러냄. 단, 검색(`ILIKE`)이 불가능해지므로 **encrypted search index** 또는 **별도 해시 컬럼**이 필요.

Q10. 거래 페이지 성능 최적화 포인트

답변

1. **`loading.tsx` Skeleton**: 각 페이지에 Suspense boundary → 데이터 로딩 중 뼈대 UI 즉시 표시
2. **`Promise.all` 병렬 조회** (`parse-core.ts:157`): 카테고리 + 계정을 동시에 조회 (직렬이면 2배 느림)

3. **`originalInput` 복호화 스킵** (`transaction.ts:372`): 리스트 조회 시 암호화된 `originalInput` 을 복호화하지 않음 → 복호화 비용 절약
4. **`optimizePackageImports`** (`next.config.ts`): lucide, recharts, motion의 tree-shaking 강화 → 번들 크기 감소
5. **`immutable` 캐시 헤더**: 정적 에셋에 `max-age=31536000, immutable` → 브라우저가 1년간 재요청 안 함
6. **Turbopack**: 개발 서버 HMR (Hot Module Replacement) 10배 빠름

꼬리: 리스트에서 일부러 안 읽는 필드?

```
originalInput: null, // 리스트 조회 시 복호화 스킵 (성능)
```

`originalInput` 은 사용자의 원래 입력 텍스트인데, AES-256-GCM 복호화는 행 수만큼 반복. 100건이면 100번 crypto 연산. 리스트에서는 이 값이 필요 없으므로 `null` 로 고정해서 성능을 확보.

Q11. 웹 경로와 API 경로의 파싱 동작이 같은가?

답변

같다. 둘 다 `parse-core.ts` 의 `executeTextParse()` / `executeImageParse()` 를 호출한다.

```
[Server Action] parseUnifiedInput() → executeTextParse()
[API Route]     POST /api/parse      → executeTextParse()
```

`parse-core.ts` 를 공용 서비스로 분리한 이유가 바로 **로직 중복 방지**. 인증 방식만 다르다:

- Server Action: `headers()` → `auth.api.getSession()`
- API Route: `request.headers` → `auth.api.getSession()`

꼬리: 다를 가능성?

현재 `parse-core.ts` 를 공유하므로 동일하지만, 만약 한쪽만 수정하면 분기할 수 있다. 예: API Route에만 rate limiting을 추가하고 Server Action에는 안 넣으면, 외부 호출은 제한되지만 앱 내부 호출은 무제한이 되는 비대칭 발생.

Q12. 인증과 초기 사용자 세팅 연결

답변 (`src/server/auth.ts:29-43`)

```
databaseHooks: {
    user: {
        create: {
            after: async (user) => {
                await db.insert(categories).values(
                    DEFAULT_CATEGORIES.map((cat, index) => ({
                        userId: user.id,
                        name: cat.name, icon: cat.icon, type: cat.type,
                        sortOrder: index + 1, isDefault: true,
                    })),
                ).onConflictDoNothing();
            },
        },
    },
},
```

회원가입 직후 12개 기본 카테고리를 자동 시딩하는 이유:

- 빈 카테고리 목록으로 시작하면 LLM이 카테고리를 매칭할 수 없음
- 사용자가 첫 거래를 입력하기 전에 "식비, 교통, 카페, 쇼핑..." 등이 이미 존재해야 함

꼬리: 실패 시 문제?

Better Auth의 `databaseHooks.user.create.after` 는 **사용자 생성 트랜잭션 밖에서** 실행된다. 즉:

- 사용자는 생성됐지만 카테고리 시딩이 실패할 수 있음 → 빈 카테고리 상태
- `onConflictDoNothing()` 이므로 재시도 시 안전 (중복 생성 안 됨)
- 현재 복구 방법: 없음. 사용자가 직접 카테고리를 추가하거나, 관리자가 수동 시딩해야 함
- **개선 가능**: 거래 페이지 로딩 시 "카테고리가 0개면 기본 시딩" 체크 로직 추가

Q13. 가장 먼저 보강하고 싶은 테스트

답변

레벨	대상	이유
Unit	`ood-filter.ts` , `bank-message.ts` , `crypto.ts`	순수 함수, 입출력 명확, 의존성 없음
Integration	`createTransactions()` (DB + 잔액 정합성)	트랜잭션 내 복잡한 로직, 가장 버그 위험
E2E	로그인 → 자연어 입력 → 확인 → 저장 → 목록 확인	핵심 사용자 플로우

회귀 위험 가장 큰 시나리오 3개

1. **계좌 잔액 정합성 깨짐**: 거래 수정 시 이전 계좌 역산 + 새 계좌 반영의 순서가 바뀌거나 누락되면 잔액이 어긋남
2. **OOD 필터 과잉 차단**: 키워드를 추가/삭제하면서 정당한 입력을 막게 됨 (예: "당근마켓 판매 30000" → "당근" 키워드 제거 시 차단)
3. **카테고리 자동 생성 실패**: `onConflictDoNothing` 로직 수정 시 unique constraint 위반으로 전체 거래 저장 실패

Q14. 1주만 개선한다면 무엇을 먼저?

답변

우선순위	개선	사용자 영향	운영 리스크	난이도
🔴 1순위	E2E 테스트 추가 (Playwright)	간접 (안정성)	높음 (회귀 방지)	중
🔴 2순위	API Rate Limiting	직접 (비용 폭주 방지)	높음 (LLM 비용)	낮
🟡 3순위	이미지 클라이언트 압축	직접 (업로드 속도)	낮	중
🟡 4순위	카테고리 시딩 보강	직접 (신규 사용자)	중	낮
🟢 5순위	description 암호화	간접 (보안)	낮	높

"당장 고쳐야" vs "나중에 해도 됨" 구분 기준

당장: 데이터 정합성이 깨지거나, 비용이 무제한 소모되거나, 사용자가 핵심 기능을 못 쓰는 경우. → Rate Limiting 없으면 누군가 API를 1만 번 호출해서 LLM 비용 폭주 가능.
나중: 성능 개선, 보안 강화, 코드 품질은 중요하지만 "지금 안 해도 서비스는 돌아간다". → description 암호화는 보안에 좋지만 검색 기능이 깨지므로 대안 설계가 먼저 필요.